



```
void SetProjectGoal()
```

```
{
```

단순 렌더링을 넘어 “UE”의 설계 방식을 따르는 엔진 아키텍처를 구축한다;

UStaticMesh 중심의 UAsset 파이프라인 및 다중 Viewport 편집 환경을
제공한다;

```
}
```

```
class Engine;
```

```
class Engine
{
    public:
        void SetEngineCore()
        {
            TObjectIterator를 통해 모든 UStaticMesh를 실시간으로 탐색하여 DropList에 자동 바인딩하는 구조;
            *.obj 및 *.mtl 파일 파싱 및 엔진 최적화 데이터로의 변환;
            매번 파싱하지 않기 위한 BinaryBaking 구축;
        }
        void Render()
        {
            위치(P), 법선(N), 색상(C), UV(T)를 포함한 정점 구조와 이를 처리하는 “StaticMeshShader.hlsl”;
            하나의 UMesh가 여러 개의 Section을 가지는 구조;
            UMesh를 Section 단위로 분할 렌더링하여 다중 UMaterial을 할당하는 로직;
        }
};
```

```
class Editor;
```

```
class Editor
{
    public:
        void SetMultiViewport()
        {
            멀티 Viewport로 화면 n분할 및 실시간 크기 조절 기능;
            SSplitter : SWindow 상속 구조와 “Editor.ini”를 통한 SSplitter 위치 저장/복원 로직;
        }
        void SceneSerialize() // 씬 관리
        {
            *.Scene 포맷을 이용한 월드 상태 저장;
            UStaticMeshComponent의 UAsset 경로 및 Transform 저장;
            Viewport의 카메라 기능 복원;
        }
};
```

```
class AdvancedFeatures;
```

```
class AdvancedFeatures
{
public:
    void SetObjViewer()
    {
        IS_OBJ_VIEWER 전처리 매크로를 사용하여 Editor와 Viewer를 물리적으로 다른 *.exe로 빌드;
        // 가벼운 뷰어 실행 파일로 인해 뷰어 파일의 뛰어난 배포성, 런타임 성능 및 리소스 관리 향상
    }
    void SetOverlayStat()
    {
        “stat” 콘솔 명령어 및 실시간 효과;
        “stat fps”, “stat memory”를 통한 엔진 리소스 실시간 모니터링 UI;
    }
}
```



```
void SetUVScroll()  
{  
    EngineLoop에서 매 프레임 계산된 DeltaTime을 누적하여 절대 시간인 TotalTime 생성;  
    연산 효율을 위해 PixelShader가 아닌 VertexShader에서 UVOffset 계산;  
    실시간으로 ScrollSpeed를 사용자가 조절할 수 있는 ScrollBarUI 구현;  
}  
};
```

```
return Showcase;
```